

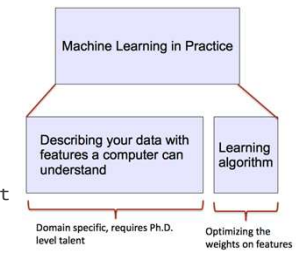
Deeplearning & Tensorflow

SHAILESH S

MACHINE VS DEEP LEARNING

- Most machine learning methods work well because of **human-designed representations** and **input features**

- ML becomes just **optimizing weights** to best make a final prediction



3

DEEP LEARNING

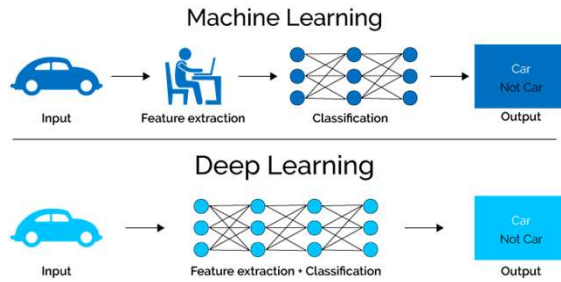


MACHINE VS DEEP LEARNING

- A machine learning subfield of learning **representations** of data. Exceptional effective at **learning patterns**.
- Deep learning algorithms attempt to learn (multiple levels of) representation by using a **hierarchy of multiple layers**
- If you provide the system **tons of information**, it begins to understand it and respond in useful ways.

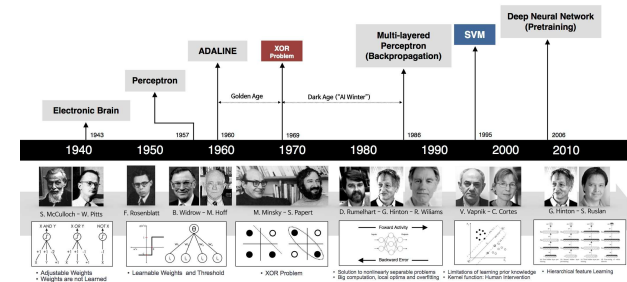
4

MACHINE VS DEEP LEARNING



5

HISTORY OF DEEP LEARNING



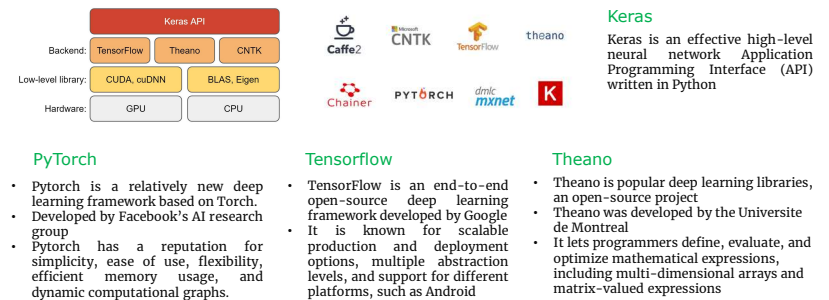
7

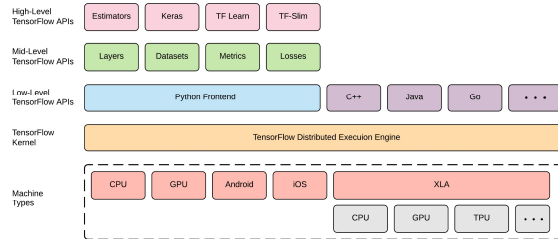
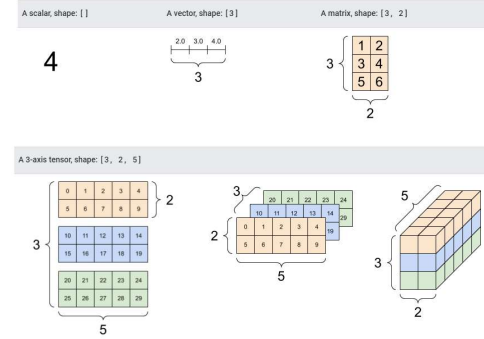
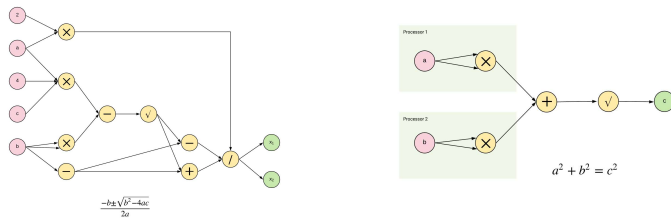
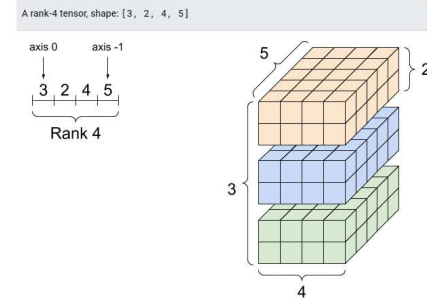
WHY DEEP LEARNING USEFUL?

- Manually designed features are often **over-specified**, **incomplete** and take a **long time to design** and validate
- Learned Features are **easy to adapt**, **fast to learn**
- Deep learning provides a very **flexible**, (almost?) **universal**, learnable framework for representing world, visual and linguistic information.
- Can learn both unsupervised and supervised
- Effective **end-to-end** joint system learning
- Utilize large amounts of training data

6

Deeplearning Technology Stack



Tensorflow**Tensors****Tensorflow Computational Graphs****Tensors**

Tensors

This will be an int32 tensor by default; see "dtypes" below.

```
rank_0_tensor = tf.constant(4)
print(rank_0_tensor)
```

Let's make this a float tensor.

```
rank_1_tensor = tf.constant([2.0, 3.0, 4.0])
print(rank_1_tensor)
```

If you want to be specific, you can set the dtype (see below) at creation time

```
rank_2_tensor = tf.constant([[1, 2],
                             [3, 4],
                             [5, 6]], dtype=tf.float16)
print(rank_2_tensor)
```

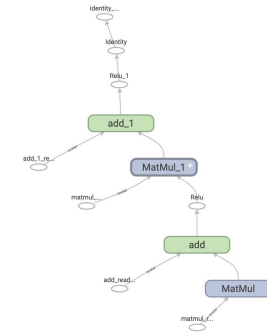
Tensorflow Graphs

Eagerly Execution means TensorFlow operations are executed by Python, operation by operation, and returning results back to Python.

Graph execution means that tensor computations are executed as a *TensorFlow graph*, sometimes referred to as a [tf.Graph](#) or simply a "graph."

While eager execution has several unique advantages, graph execution enables portability outside Python and tends to offer better performance

Graphs are data structures that contain a set of [tf.Operation](#) objects, which represent units of computation; and [tf.Tensor](#) objects, which represent the units of data that flow between operations. They are defined in a [tf.Graph](#) context.



Tensors

There can be an arbitrary number of
axes (sometimes called "dimensions")

```
rank_3_tensor = tf.constant([
    [0, 1, 2, 3, 4],
    [5, 6, 7, 8, 9]],
    [[10, 11, 12, 13, 14],
     [15, 16, 17, 18, 19]],
    [[20, 21, 22, 23, 24],
     [25, 26, 27, 28, 29]],])
print(rank_3_tensor)
```

Variables

```
my_tensor = tf.constant([[1.0, 2.0], [3.0, 4.0]])
```

```
my_variable = tf.Variable(my_tensor)
```

Variables can be all kinds of types, just like tensors

```
bool_variable = tf.Variable([False, False, False, True])
```

```
complex_variable = tf.Variable([5 + 4j, 6 + 1j])
```

THANK YOU